

STABLECOIN CHECKOUT

Arcora

Customer pays with what they have. Merchant settles in the stablecoin they want.
Atomically, on Arc.

v1.0.2 · live on Arc testnet · 117 contract tests passing

The problem

Stablecoins are now **everywhere** — USDC alone moved \$5T in 2025. But for a merchant accepting them, payment is still:

- Multiple chains
- Multiple stables (USDC, EURC, USDT, PYUSD, DAI, regional pegs...)
- Custodial off-ramps that take days and 1.5%
- Or a thicket of bridges and approvals every customer has to learn

"I have USDC on Arbitrum. The merchant wants EURC on Arc. Five clicks and twenty minutes later I'm not sure if my money got there."

What Arcora is

A **Stripe-like checkout for stablecoin payments**, built around one bet:

Customer pays from where they are, with whatever they have.

Merchant receives their preferred stablecoin on Arc.

One on-chain transaction. One signature surface for the customer. The merchant never has to think about the customer's chain, their token, or the FX in between.

Live demo — try it now

Demo merchant: arc-fx-demo.vercel.app · "Acme Coffee, €4.50"

Hosted checkout app: arc-fx-gateway.vercel.app

Merchant dashboard: </m/dashboard> · </m/treasury>

On-chain: Arc testnet · Gateway `0x7c1137...F25F208Fb7a3`

```
npm install @arcora/sdk
```

How it works — three steps

01	Merchant creates an invoice	3-line SDK call or 1 click in the dashboard
02	Customer pays at the checkout	Connect wallet, approve, pay — one tx
03	Settles on Arc, atomically	Swap (if needed) + payout + fee + event, in a single tx

```
import { Arcora } from "@arcora/sdk";

Arcora.init({ apiKey });
const inv = await Arcora.createInvoice({ amountUsdc: 49.99, payInToken: "EURC" });
Arcora.openCheckout(inv);
```

What's live in v1.0.2

-  **Hosted checkout** with SIWE merchant auth, dashboard, QR sharing
-  **Atomic FX settlement** via Chainlink-priced OracleAMM (USDC $\rightleftharpoons$ EURC)
-  **Same-token fast path** when payIn = payout (no swap, no slippage)
-  **Refunds** — `refundInvoice()` returns the customer's payout + the protocol fee back to the merchant from accrued
-  **Treasury dashboard** — per-stable net received, gross volume, refunded, fees, activity feed
-  **HMAC-signed webhooks** — `invoice.paid`, `invoice.refunded`, retry with exponential backoff
-  **Two npm packages** — `@arcora/sdk`, `@arcora/sdk-react`

The killer-feature bet

Layer	v1.0	v2.0	v3.0
Customer's chain	Arc	Any CCTP-supported EVM	Any chain (Solana, Sui...)
Customer's token	USDC, EURC	+ USDT, PYUSD, DAI, regional	+ native ETH, any ERC-20
Signatures	1–2	2–3	1 intent
Merchant settles in	USDC or EURC on Arc	Any supported stable on Arc	Same

We are at v1.0 today. v2.0 — crosschain via CCTP — is the differentiator other stablecoin processors can't ship without rebuilding their stack.

Architecture

```
Customer wallet -(approve + pay)→ ArcFXGateway (immutable)
|
├ same-token? → direct transfer
|
└ swap? → OracleAMM (Chainlink-priced ± 4 bps)
        |
        ├── payout token → merchant
        └ fee accrued for refund pool
|
emit InvoicePaid

VPS daemons    → index events 30s    → Neon Postgres mirror
                → HMAC webhook 10s   → merchant endpoint
```

Gateway, AMM, oracle, indexer, webhook dispatcher, hosted app, SDK — all in one repo, one team, one deploy graph.

Arcora sits **above** Arc primitives

Arc itself ships first-party financial rails. Arcora is the **merchant abstraction layer** on top — same shape Stripe Checkout has on top of card-network rails.

Concern	Arc primitive	Arcora
Enterprise FX (RFQ + escrow)	StableFX	—
Generic A→B swap	App Kit Swap	—
Crosschain USDC bridge	App Kit Bridge	—
Server wallet management	Circle Developer-Controlled Wallets	—
Compliance (Elliptic / TRM Labs)	App Kit hooks	—
Invoice → atomic settle in exact payout token	—	ArcFXGateway
Hosted checkout link + customer wallet flow	—	<code>/i/[invoiceId]</code>
Refund-in-payout-token + protocol-fee return	—	<code>refundInvoice()</code>
Per-merchant treasury reconciliation	—	<code>/m/treasury</code>

Proof — testnet metrics

Surface	Number
Foundry tests passing	117 (unit + fuzz 10k runs + invariant 256×64)
Vitest tests passing	46 (auth, crypto, schema, API routes, components)
Live txs end-to-end	4 distinct flows (same-token + swap, both with refund)
Contract bytecode	2.1M gas to deploy (well under block limit)
Deploy cost	< \$0.10 USDC (Arc settles gas in USDC, not ETH)

Sample on-chain proofs:

- Same-token pay: 0x3f2fc3ff...84ef08
- EURC→USDC swap pay: 0xa35cdab6...45ae8
- Refund (swap): 0x2dc24ed9...733cc42

Why Arc, why now

Arc is Circle's stablecoin-native L1 — purpose-built for the rails Arcora needs:

- **USDC is the gas token** — no native ETH friction
- **CCTP V2 destination domain** 26 is live → any CCTP source chain bridges natively
- **Stablecoin-first economy** by design — fee mechanics + sequencer policy assume payment volume, not memecoin churn

Now because:

- Stablecoin payment volume crossed \$50B/month in 2025
- Circle Gateway + CCTP V2 unlocked native unified balances across chains
- Arc testnet is open + supported, mainnet on the horizon

We are early enough that **which** stablecoin checkout becomes the default on Arc is still up for grabs.

Roadmap

v1.0 LIVE	Arc-only USDC ⇔ EURC + refunds + treasury + npm SDK	shipped 2026-04-29
v1.x	Any stablecoin on Arc (USDT, PYUSD, DAI, regional)	spec drafted
v2.0	Crosschain USDC source via CCTP (Eth, Arb, Base, OP, Polygon, Avalanche, Linea, Codex)	next quarter
v2.1	Source-side DEX aggregator — pay with native ETH or any ERC-20	following
v2.2	Solana / Sui / non-EVM	following
v3.0	One-signature intent solver — full Stripe-like UX	endgame

Each row builds on the previous one without a rewrite. Shipping v1 was the proof.

Pre-mainnet bars

Before Arcora goes live on Arc mainnet:

- **Real Chainlink price feeds** (replace `MockChainlinkFeed` testnet pattern; oracle-keepalive timer becomes obsolete)
- **External security audit** + Slither / Mythril gate
- **KYB / merchant onboarding** flow (testnet is wallet-only)
- **Domain & branding:** `arcorapay.com` registered + `checkout.` / `dashboard.` / `docs.` subdomains
- **Treasury reserve policy** (refund float, fee-withdrawal cadence)

These are scoped, not exploratory. Each one is a 1–2 week task.

Ask

For **builders / partners**:

- Try the demo, integrate the SDK, file issues
- `npm install @arcora/sdk` — it works today
- GitHub: github.com/Kubudak90/arc-fx-gateway

For **investors / Arc ecosystem**:

- v1 ships on testnet **today**. v2 (crosschain) is the wedge.
- Looking for: ecosystem support on Arc mainnet, pre-seed for KYB + audit + domain spend, distribution partners for hosted-checkout.

For **everyone**:

- The customer-side problem ("right token, wrong chain") is real. Arcora's bet is that the answer is **one checkout, not five steps**.

DEMO · Q&A

Pay anywhere. Settle on Arc.

arcora.dev (soon) · arc-fx-gateway.vercel.app · @arcora/sdk

Hüseyin · 2026